Table Optimisation

Summary

Redshift table design focuses on optimizing distribution styles, sort keys, and compression encoding to improve query performance and reduce storage cost. Redshift can manage these automatically using AUTO settings, or users can manually configure them based on workload and query patterns.

Automatic Table Optimization

Redshift automatically optimizes tables using AI-based analysis.

Automatically Managed Areas

Feature	Purpose
Distribution Keys	Balance data across nodes
Sort Keys	Optimize query scanning
Compression	Reduce storage and improve performance

You can also manually customize these settings if needed.

Distribution Styles

Distribution style controls how data is distributed across nodes and slices.

Main Goals

- Uniform workload distribution
- Reduce data movement during queries

---

1. ALL Distribution

Behavior

Entire table is copied to every node.

Best For

- Small tables
- Slowly changing tables

Advantage

- No redistribution during joins

Drawback

- Slower inserts and updates
- Higher storage usage

```
DISTSTYLE ALL
```

2. EVEN Distribution

Behavior

Rows distributed equally using round-robin method.

Best For

- Single table queries
- Balanced storage

Drawback

- Joins may require expensive redistribution

```
DISTSTYLE EVEN
```

3. KEY Distribution

Behavior

Rows distributed using values from one column.

Example

```
userid
city
buyerid
```

Rows with same key values are stored together.

Best For

- Frequently joined tables
- Predicate-heavy queries

Example

- `sales.buyerid`
- `users.userid`

Both tables can be colocated for faster joins.

```
DISTSTYLE KEY
DISTKEY(userid)
```

4. AUTO Distribution

Behavior

Redshift automatically selects:

- ALL

- EVEN
- KEY

based on:

- Table size
- Query patterns
- Performance analysis

Advantage

- Fully managed optimization

Drawback

- Less manual control

```
DISTSTYLE AUTO
```

Change Distribution Style

ALTER TABLE Example

```
ALTER TABLE users
ALTER DISTSTYLE KEY
DISTKEY(userid);
```

Choosing Good Distribution Styles

Before selecting manually:

- Analyze costly queries
- Identify join columns
- Review predicates and aggregates
- Check query execution plans

Important

Distribution should be decided based on:

- Entire workload
- Query behavior
- Multiple related tables

Not just one table alone.

Sort Keys

Sort keys improve:

- Data scanning
- Read performance

- Query efficiency

AUTO Sort Key

If not specified:

```
SORTKEY AUTO
```

Redshift automatically manages sorting.

Manual Sort Key

Single Column Sort Key

```
firstname VARCHAR SORTKEY
```

Compound Sort Key

Default sort key type.

```
SORTKEY(firstname, lastname)
```

Interleaved Sort Key

Useful when multiple columns are queried equally.

```
INTERLEAVED SORTKEY(city, state)
```

Limits

Maximum:

```
400 sort key columns
```

per table.

Compression Encoding

Compression reduces:

- Storage usage
- Disk I/O
- Query execution cost

Common Compression Types

Encoding	Usage

RAW	No compression
AZ64	Numbers, dates, timestamps
LZO	CHAR / VARCHAR
BYTEDICT	Repeated values
DELTA	Sequential values

Automatic Compression

Default:

```
ENCODE AUTO
```

Redshift automatically selects compression type.

Automatic Compression Rules

AZ64 Used For

- SMALLINT
- INTEGER
- BIGINT
- DECIMAL
- DATE
- TIMESTAMP

LZO Used For

- CHAR
- VARCHAR

RAW Used For

- Sort key columns
- BOOLEAN
- REAL
- DOUBLE PRECISION

Manual Compression Example

```
CREATE TABLE users (
    userid INTEGER ENCODE az64,
    firstname VARCHAR ENCODE lzos
);
```

Analyze Compression

Used to identify better compression options.

```
ANALYZE COMPRESSION users;
```

Key Learning

Amazon Redshift can automatically optimize:

- Data distribution
- Sorting
- Compression

But once you manually configure these settings, you become responsible for managing and optimizing them in the future.
